T3000 API & Protocol

Configuration Guide · built Nov 13, 2026

Configuration Guide

Learn how to configure T3000 for your building automation needs.

Initial Configuration

After installation, configure these essential settings.

Network Settings

Navigate to **Settings → Network**

IP Configuration

```text
Network Adapter: [Select your adapter]
IP Address Range: 192.168.1.1 - 192.168.1.254
Subnet Mask: 255.255.255.0
Gateway: 192.168.1.1
```

Protocol Settings

- ✓ Enable Modbus TCP (Port 502)
- ✓ Enable BACnet (Port 47808)
- ○ Enable HTTP API (Port 8080)

Database Settings

Location

```text
Default: C:\ProgramData\Temco\T3000\Database
```

Backup

- Enable automatic backups: ✓

- Backup frequency: Daily at 2:00 AM
- Keep backups for: 30 days
- Backup location: `C:\Backups\T3000`

User Management

Create user accounts with appropriate permissions.

Administrator Account

- Full access to all features
- Can create/modify users
- Can change system settings

Operator Account

- View and control data points
- Cannot modify device settings
- Cannot add/remove devices

Viewer Account

- Read-only access
- Can view dashboards and trends
- Cannot make changes

Communication Protocols

Modbus Configuration

Modbus TCP/IP

```yaml
Protocol: Modbus TCP
Port: 502
Timeout: 5000 ms
Retries: 3
Poll Rate: 1000 ms
```

Modbus RTU (Serial)

```yaml
Port: COM1
Baud Rate: 19200
Data Bits: 8
Parity: None
Stop Bits: 1
```

BACnet Configuration

```yaml
Protocol: BACnet/IP
Port: 47808
Device Instance: 389001
Network Number: 0
Max APDU: 1476
```

System Preferences

Display Settings

- **Temperature Units**: °F / °C
- **Time Format**: 12-hour / 24-hour
- **Date Format**: MM/DD/YYYY
- **Decimal Places**: 1

Alarm Settings

Email Notifications

```
SMTP Server: smtp.gmail.com
Port: 587
Username: alerts@yourdomain.com
Use TLS: ✓
```

Alert Priorities

- Critical: Immediate notification
- High: Within 5 minutes
- Medium: Within 15 minutes

- Low: Daily summary

Data Logging

Trend Log Settings

- Sample interval: 1 minute (adjustable)
- Storage period: 1 year
- Auto-archive: Enabled
- Compression: Enabled

Security Settings

Password Policy

- Minimum length: 8 characters
- Require uppercase: ✓
- Require numbers: ✓
- Require symbols: ○
- Password expiry: 90 days

Session Settings

- Timeout: 30 minutes
- Concurrent logins: Allowed
- Remember last login: ✓

Performance Optimization

Polling Optimization

Balance between real-time updates and network load:

Device Count	Recommended Poll Rate
1-10	1000 ms
11-50	2000 ms
51-100	5000 ms
100+	10000 ms

Database Maintenance

Schedule regular maintenance:

- **Compact database**: Weekly
- **Rebuild indexes**: Monthly
- **Clear old logs**: Quarterly

Exporting/Importing Configuration

Export Settings

1. Go to **Settings** → **System**
2. Click **Export Configuration**
3. Save as `T3000_Config_Backup.json`

Import Settings

1. Go to **Settings** → **System**
2. Click **Import Configuration**
3. Select saved `.json` file
4. Restart application when prompted

Configuration Best Practices

✅ **Do:**

- Back up configuration regularly
- Use strong passwords
- Enable audit logging
- Test network connectivity before adding devices

❌ **Don't:**

- Use default passwords in production
- Disable security features without reason
- Set poll rates too aggressively
- Ignore backup alerts

Next Steps

- Connecting Devices
- Creating Graphics

- Setting Up Schedules

Configuration Guide

Programmatic Configuration

Configuration File Format

JSON

```
{
  "version": "9.0",
  "network": {
    "adapters": [
      {
        "name": "Ethernet",
        "ipRange": "192.168.1.0/24",
        "gateway": "192.168.1.1",
        "dns": ["8.8.8.8", "8.8.4.4"]
      }
    ],
    "protocols": {
      "modbusTcp": {
        "enabled": true,
        "port": 502,
        "timeout": 5000,
        "retries": 3,
        "slaveIds": [1, 2, 3, 4, 5]
      },
      "bacnet": {
        "enabled": true,
        "port": 47808,
        "deviceInstance": 389001,
        "networkNumber": 0,
        "maxApdu": 1476,
        "segmentation": "both",
        "apduTimeout": 3000
      },
      "http": {
        "enabled": true,
        "port": 9103,
        "cors": true,
        "apiKey": "your-api-key-here"
      }
```

```
    }
  },
  "database": {
    "path": "C:/ProgramData/Temco/T3000/t3000.db",
    "backup": {
      "enabled": true,
      "schedule": "0 2 * * *",
      "retention": 30,
      "location": "C:/Backups/T3000"
    },
    "performance": {
      "cacheSize": 10000,
      "walMode": true,
      "synchronous": "NORMAL"
    }
  },
  "security": {
    "passwordPolicy": {
      "minLength": 8,
      "requireUppercase": true,
      "requireNumbers": true,
      "requireSymbols": false,
      "expiryDays": 90
    },
    "session": {
      "timeout": 1800,
      "allowConcurrent": true,
      "rememberLastLogin": true
    },
    "audit": {
      "enabled": true,
      "logLevel": "INFO",
      "retention": 365
    }
  },
  "performance": {
    "pollRate": 1000,
    "maxConcurrentRequests": 10,
    "requestTimeout": 30000,
    "cacheEnabled": true
```

```
  }
}
```

Loading Configuration

TYPESCRIPT

```typescript
import { loadConfig, validateConfig } from '@temco/t3000-config';

// Load configuration from file
const config = await loadConfig('./config.json');

// Validate configuration
const validation = validateConfig(config);
if (!validation.valid) {
  console.error('Invalid configuration:', validation.errors);
  process.exit(1);
}

// Apply configuration
await system.applyConfig(config);
```

Environment-Based Configuration

TYPESCRIPT

```typescript
// config/development.ts
export const devConfig = {
  network: {
    protocols: {
      modbusTcp: { port: 5020 },  // Non-standard port for dev
      bacnet: { port: 47809 },
      http: { port: 3003 }
    }
  },
  database: {
    path: './dev-data/t3000.db'
  },
  logging: {
    level: 'DEBUG'
  }
};

// config/production.ts
export const prodConfig = {
  network: {
    protocols: {
      modbusTcp: { port: 502 },
      bacnet: { port: 47808 },
      http: { port: 9103 }
    }
  },
  database: {
    path: 'C:/ProgramData/Temco/T3000/t3000.db'
  },
  logging: {
    level: 'INFO'
  }
};

// Load based on environment
const config = process.env.NODE_ENV === 'production'
  ? prodConfig
  : devConfig;
```

API-Based Configuration

REST API Configuration

SHELL

```shell
## Get current configuration
curl http://localhost:9103/api/config

## Update network settings
curl -X PATCH http://localhost:9103/api/config/network \
  -H "Content-Type: application/json" \
  -d '{
    "protocols": {
      "modbusTcp": {
        "enabled": true,
        "port": 502,
        "timeout": 5000
      }
    }
  }'

## Update database settings
curl -X PATCH http://localhost:9103/api/config/database \
  -H "Content-Type: application/json" \
  -d '{
    "backup": {
      "enabled": true,
      "schedule": "0 2 * * *"
    }
  }'
```

Python SDK Configuration

PYTHON

```python
from t3000 import T3000Client, Config

## Connect to T3000 instance
client = T3000Client(host='localhost', port=9103)

## Get current configuration
config = client.get_config()

## Modify configuration
config.network.protocols.modbus_tcp.port = 502
config.network.protocols.bacnet.port = 47808

## Apply configuration
client.set_config(config)

## Or update specific section
client.update_config('network.protocols.modbusTcp', {
    'enabled': True,
    'port': 502,
    'timeout': 5000
})
```

Database Schema Configuration

SQLite Optimization

SQL

```sql
-- Performance pragmas
PRAGMA journal_mode = WAL;
PRAGMA synchronous = NORMAL;
PRAGMA cache_size = 10000;
PRAGMA temp_store = MEMORY;
PRAGMA mmap_size = 30000000000;

-- Create indexes for common queries
CREATE INDEX IF NOT EXISTS idx_devices_serial
  ON devices(serial_number);

CREATE INDEX IF NOT EXISTS idx_points_device_type
  ON data_points(device_id, point_type);

CREATE INDEX IF NOT EXISTS idx_trends_device_time
  ON trend_data(device_id, timestamp DESC);

-- Analyze for query optimization
ANALYZE;
```

Database Migrations

TYPESCRIPT

```typescript
import Database from 'better-sqlite3';

interface Migration {
  version: number;
  up: string;
  down: string;
}

const migrations: Migration[] = [
  {
    version: 1,
    up: `CREATE TABLE devices (...)`,
    down: `DROP TABLE devices`
  },
  {
    version: 2,
    up: `ALTER TABLE devices ADD COLUMN firmware_version TEXT`,
    down: `ALTER TABLE devices DROP COLUMN firmware_version`
  }
];

function runMigrations(db: Database.Database) {
  // Create migrations table
  db.exec(`
    CREATE TABLE IF NOT EXISTS migrations (
      version INTEGER PRIMARY KEY,
      applied_at INTEGER
    )
  `);

  // Get current version
  const current = db.prepare(
    'SELECT MAX(version) as v FROM migrations'
  ).get() as { v: number };

  const currentVersion = current?.v || 0;

  // Apply pending migrations
```

```
  for (const migration of migrations) {
    if (migration.version > currentVersion) {
      db.exec(migration.up);
      db.prepare(
        'INSERT INTO migrations (version, applied_at) VALUES (?, ?)'
      ).run(migration.version, Date.now());

      console.log(`Applied migration ${migration.version}`);
    }
  }
}
```

Protocol-Specific Configuration

BACnet Advanced Configuration

TYPESCRIPT

```typescript
interface BACnetConfig {
  deviceInstance: number;
  networkNumber: number;
  macAddress?: string;
  maxApdu: 1476 | 1024 | 480 | 206 | 128 | 50;
  segmentation: 'both' | 'transmit' | 'receive' | 'none';
  apduTimeout: number;
  numberOfApduRetries: number;
  deviceId: number;
  objectName: string;
  modelName: string;
  firmwareRevision: string;
  applicationSoftwareVersion: string;
  location: string;
  description: string;
  vendorIdentifier: number;
}

const bacnetConfig: BACnetConfig = {
  deviceInstance: 389001,
  networkNumber: 0,
  maxApdu: 1476,
  segmentation: 'both',
  apduTimeout: 3000,
  numberOfApduRetries: 3,
  deviceId: 389001,
  objectName: 'T3000-Web-Server',
  modelName: 'T3000',
  firmwareRevision: '9.0.0',
  applicationSoftwareVersion: '9.0.0',
  location: 'Building 1',
  description: 'T3000 BACnet Gateway',
  vendorIdentifier: 373
};
```

Modbus Register Mapping

TYPESCRIPT

```typescript
interface ModbusRegisterMap {
  startAddress: number;
  count: number;
  pointType: string;
  conversion: (raw: number) => number;
}

const registerMaps: Record<string, ModbusRegisterMap> = {
  inputs: {
    startAddress: 0,
    count: 64,
    pointType: 'AI',
    conversion: (raw) => raw / 10.0  // Convert to engineering units
  },
  outputs: {
    startAddress: 100,
    count: 32,
    pointType: 'AO',
    conversion: (raw) => raw / 10.0
  },
  variables: {
    startAddress: 200,
    count: 200,
    pointType: 'VAR',
    conversion: (raw) => raw / 10.0
  }
};
```

Configuration Validation

TYPESCRIPT

```typescript
import Ajv from 'ajv';

const configSchema = {
  type: 'object',
  required: ['network', 'database'],
  properties: {
    network: {
      type: 'object',
      required: ['protocols'],
      properties: {
        protocols: {
          type: 'object',
          properties: {
            modbusTcp: {
              type: 'object',
              properties: {
                port: { type: 'number', minimum: 1, maximum: 65535 },
                timeout: { type: 'number', minimum: 1000 },
                retries: { type: 'number', minimum: 0, maximum: 10 }
              }
            }
          }
        }
      }
    },
    database: {
      type: 'object',
      required: ['path'],
      properties: {
        path: { type: 'string', minLength: 1 }
      }
    }
  }
};

const ajv = new Ajv();
const validate = ajv.compile(configSchema);
```

```
function validateConfiguration(config: unknown): boolean {
  const valid = validate(config);
  if (!valid) {
    console.error('Configuration validation errors:', validate.errors);
  }
  return valid;
}
```

Next Steps

- REST API Reference
- Device API
- Database Schema

Initial Configuration

After installation, configure these essential settings.

Network Settings

Navigate to **Settings** → **Network**

IP Configuration

```
Network Adapter: [Select your adapter]
IP Address Range: 192.168.1.1 - 192.168.1.254
Subnet Mask: 255.255.255.0
Gateway: 192.168.1.1
```

Protocol Settings

- ✓ Enable Modbus TCP (Port 502)
- ✓ Enable BACnet (Port 47808)
- ○ Enable HTTP API (Port 8080)

Database Settings

Location

```
Default: C:\ProgramData\Temco\T3000\Database
```

Backup

- Enable automatic backups: ✓
- Backup frequency: Daily at 2:00 AM
- Keep backups for: 30 days
- Backup location: `C:\Backups\T3000`

User Management

Create user accounts with appropriate permissions.

Administrator Account

- Full access to all features
- Can create/modify users
- Can change system settings

Operator Account

- View and control data points
- Cannot modify device settings
- Cannot add/remove devices

Viewer Account

- Read-only access
- Can view dashboards and trends
- Cannot make changes

Communication Protocols

Modbus Configuration

Modbus TCP/IP

```yaml
Protocol: Modbus TCP
Port: 502
Timeout: 5000 ms
Retries: 3
Poll Rate: 1000 ms
```

Modbus RTU (Serial)

```yaml
Port: COM1
Baud Rate: 19200
Data Bits: 8
Parity: None
Stop Bits: 1
```

BACnet Configuration

```yaml
Protocol: BACnet/IP
Port: 47808
Device Instance: 389001
Network Number: 0
Max APDU: 1476
```

System Preferences

Display Settings

- **Temperature Units**: °F / °C
- **Time Format**: 12-hour / 24-hour
- **Date Format**: MM/DD/YYYY
- **Decimal Places**: 1

Alarm Settings

Email Notifications

```
SMTP Server: smtp.gmail.com
Port: 587
Username: alerts@yourdomain.com
Use TLS: ✓
```

Alert Priorities

- Critical: Immediate notification
- High: Within 5 minutes
- Medium: Within 15 minutes

- Low: Daily summary

Data Logging

Trend Log Settings

- Sample interval: 1 minute (adjustable)
- Storage period: 1 year
- Auto-archive: Enabled
- Compression: Enabled

Security Settings

Password Policy

- Minimum length: 8 characters
- Require uppercase: ✓
- Require numbers: ✓
- Require symbols: ○
- Password expiry: 90 days

Session Settings

- Timeout: 30 minutes
- Concurrent logins: Allowed
- Remember last login: ✓

Performance Optimization

Polling Optimization

Balance between real-time updates and network load:

Device Count	Recommended Poll Rate
1-10	1000 ms
11-50	2000 ms
51-100	5000 ms
100+	10000 ms

Database Maintenance

Schedule regular maintenance:

- **Compact database**: Weekly
- **Rebuild indexes**: Monthly
- **Clear old logs**: Quarterly

Exporting/Importing Configuration

Export Settings

1. Go to **Settings** → **System**
2. Click **Export Configuration**
3. Save as `T3000_Config_Backup.json`

Import Settings

1. Go to **Settings** → **System**
2. Click **Import Configuration**
3. Select saved `.json` file
4. Restart application when prompted

Next Steps

- Connecting Devices
- Creating Graphics
- Setting Up Schedules

Configuration Best Practices

✅ **Do:**

- Back up configuration regularly
- Use strong passwords
- Enable audit logging
- Test network connectivity before adding devices

❌ **Don't:**

- Use default passwords in production
- Disable security features without reason
- Set poll rates too aggressively

- Ignore backup alerts